FederHub: System Architecture & UML Design Document

Team 4

Abstract

FederHub is a comprehensive system designed to support privacy-preserving machine learning through decentralized orchestration, secure data handling, and metric monitoring[cite: 4]. The platform is built around the specific workflows of three primary personas: the Platform Admin, the ML Engineer (Federation Owner), and the Client Operator[cite: 3]. By distributing the training processes directly to the client nodes, FederHub ensures that sensitive raw data never leaves the organization's secure, local environment. This document outlines the technical architecture, UML use cases, sequence diagrams, and sprint progress that fulfill these core privacy and management requirements.

1. System Architecture

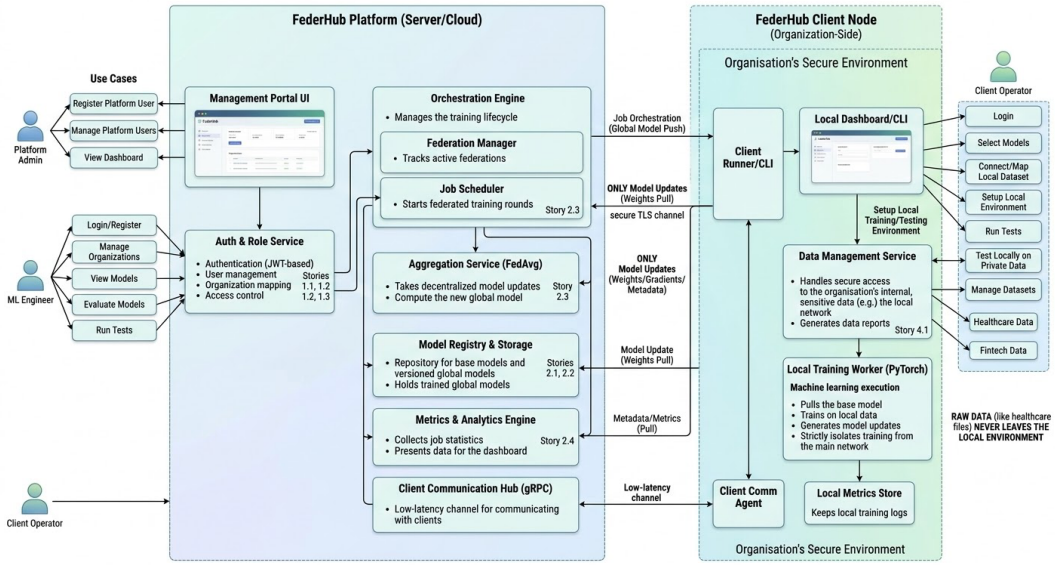


Figure 1: FederHub High-Level System Architecture

Architecture & User Story Alignment:

- **Privacy by Design (Epic 3, Story 3.1):** The architecture physically separates the Central Hub from the Edge Nodes, ensuring raw client data is locked securely behind local firewalls.
- **Orchestration Engine (Epic 2):** The integration of Celery and Redis provides the asynchronous task queuing required to reliably manage training rounds across multiple clients.
- **Secure Communication:** Utilizes gRPC pipelines between the Edge and Hub to facilitate the lightning-fast, encrypted streaming of mathematical model updates (Epic 4, Story 4.2).
- **Local Execution (Epic 4, Story 4.3):** Shows the desktop application spinning up isolated Docker containers to process PyTorch/TensorFlow scripts safely on the client's machine.
- **Centralized Management (Epic 1):** The FastAPI and PostgreSQL backend supports the Platform Admin's need to govern access and securely store job metadata.
- **FederHub Platform (Server/Cloud):** Contains the Management Portal UI, Auth & Role Service, Orchestration Engine (Federation Manager and Job Scheduler), Aggregation Service (FedAvg), Model Registry & Storage, and Metrics & Analytics Engine.
- **FederHub Client Node (Organization-Side):** Features a Client Runner/CLI, Local Dashboard/CLI, Data Management Service, Local Training Worker (PyTorch), and Local Metrics Store.
- **Communication Channels:** Uses a secure TLS channel for model updates and a low-latency channel (gRPC) via the Client Communication Hub.

2. Use Case Diagrams

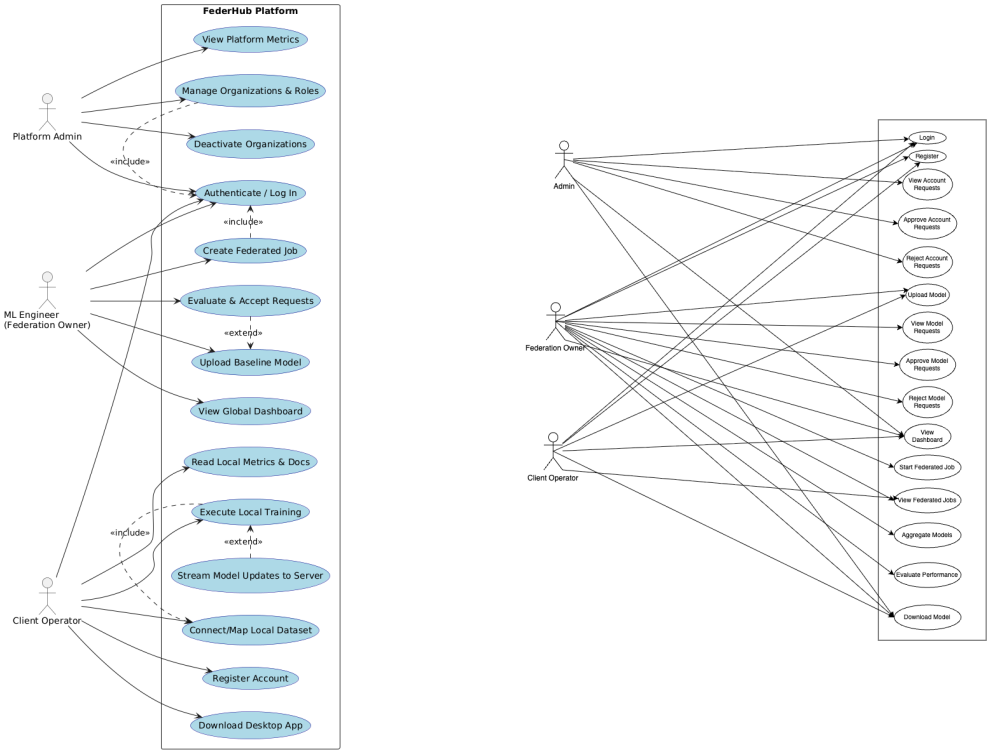


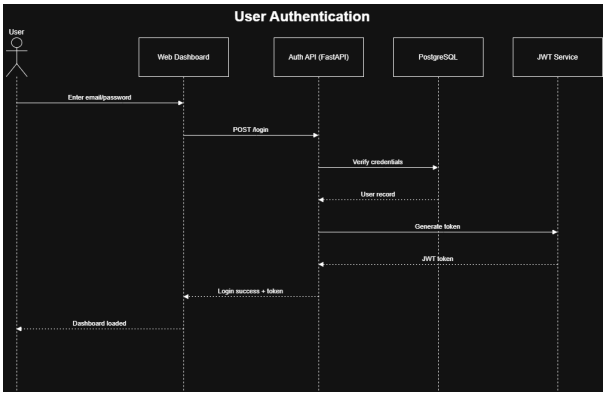
Figure 2: FederHub System Use Cases

Use Case & User Story Alignment:

- **Actor Mapping:** Clearly defines the system boundaries for the Platform Admin, ML Engineer, and Client Operator, mapping directly to the personas established in Sprint 1.
- **Feature Representation:** Every oval (Use Case) represents a direct translation of a User Story (e.g., "Connect/Map Local Dataset" directly fulfills Epic 3, Story 3.2).
- **Advanced Dependencies:** Utilizes `<<include>>` to enforce prerequisites (like requiring authentication before job creation) and `<<extend>>` to show conditional actions (like streaming weights after training completes).

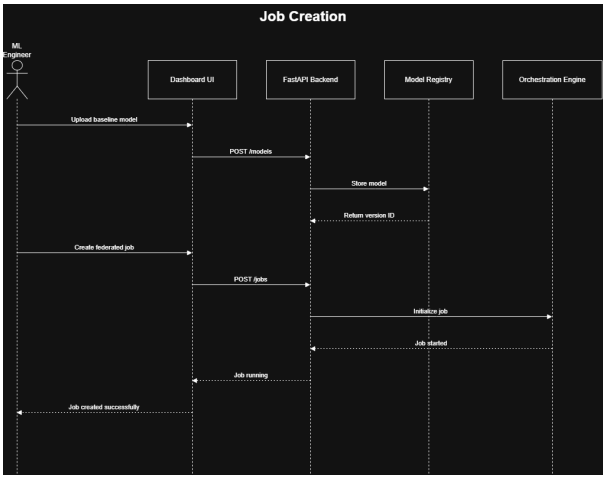
3. Interaction & Sequence Diagrams

The following sequence diagrams demonstrate the step-by-step chronological transfer of information, proving how our technical architecture executes the planned user stories.



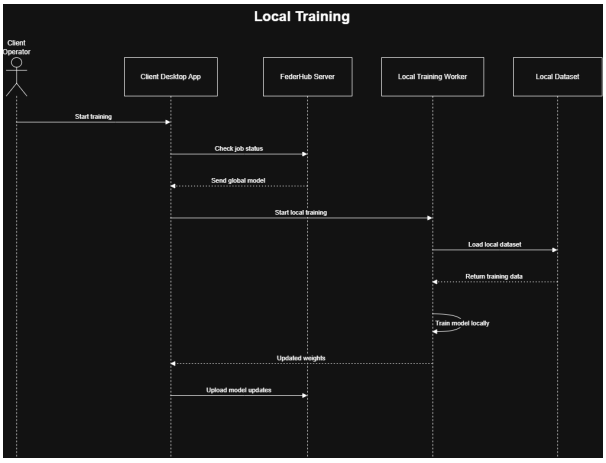
Sequence 1: User Authentication

- **Fulfills Epic 1, Story 1.3:** Validates user credentials against the PostgreSQL database.
- Generates and returns a secure JWT to ensure the client session remains authenticated.



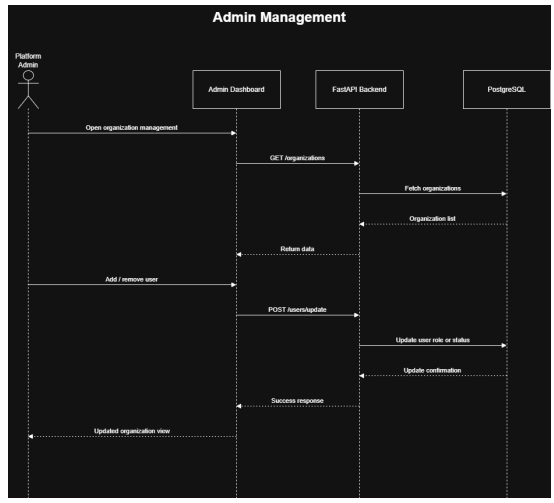
Sequence 2: Job Creation

- **Fulfills Epic 2, Story 2.3:** ML Engineer selects a baseline model and configures hyperparameters.
- Fast API validates and persists the job setup to the database, initiating the orchestration loop.



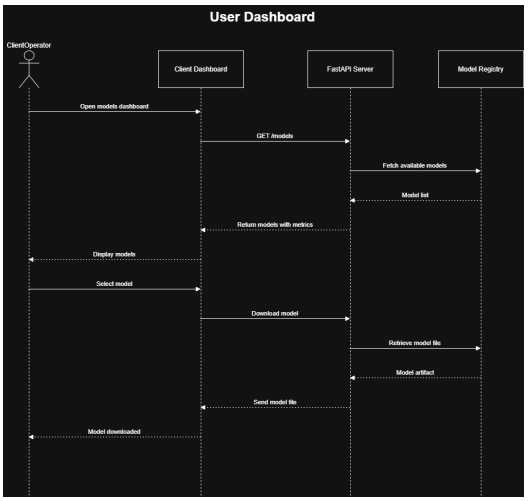
Sequence 3: Local Training

- **Realizes Epic 3, Story 3.1:** Proves the raw dataset is mounted strictly locally via Docker.
- Client trains the model and securely pushes only updated weights to the server.



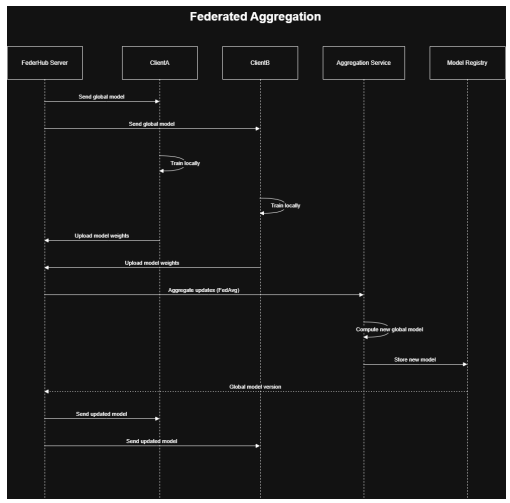
Sequence 4: Admin Management

- **Fulfills Epic 1, Story 1.1:** Details the workflow for Admins to view and approve organization requests.
- Ensures Role-Based Access Control (RBAC) is enforced at the database layer.



Sequence 5: User Dashboard

- **Corresponds to Epic 4, Story 4.1:** Client Operator queries the server for available models.
- Server fetches versioned data and performance metrics to populate the React frontend.



Sequence 6: Federated Aggregation

- **Executes Epic 2, Story 2.3:** Server collects local updates across the gRPC channel.
- Applies the FedAvg algorithm to compute the new global model, completing the lifecycle.

4. Design Decisions: Why UML?

The team chose UML for this sprint to visually map the stories noted in the architecture diagram to actual system behavior:

- **Clarity on Data Flow:** The diagrams explicitly show that Model Updates (Weights/Gradients/Metadata) are the only items sent to the server, satisfying the requirement that raw data never leaves the local environment.
- **Component Responsibility:** By using Sequence Diagrams, we defined exactly which service (e.g., FastAPI Backend vs. Orchestration Engine) handles specific API calls like `POST /jobs` or `GET /organizations`.

5. Conclusion

This document confirms that the technical architecture is fully aligned with the planned features. By using the Aggregation Service and Local Training Workers, FederHub achieves decentralized training while maintaining centralized management through the FederHub Platform UI.

6. Sprint Progress: Burndown Chart

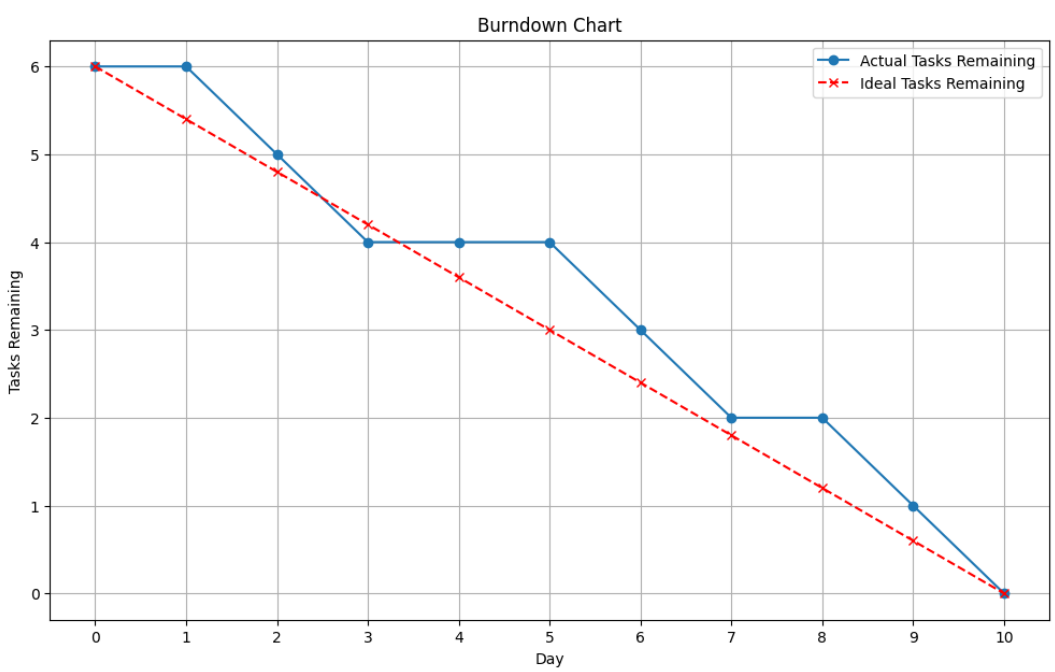


Figure 3: Sprint Burndown Chart tracking actual vs. ideal tasks remaining.